

1. Image Digitization using Sampling and Quantization

```
clc;

clear;

img = imread('image.jpg');    % Read image
gray = rgb2gray(img);        % Convert to grayscale

sampled = gray(1:2:end,1:2:end); % Sampling
quantized = uint8(gray/16)*16; % Quantization

subplot(1,3,1), imshow(gray), title('Original');
subplot(1,3,2), imshow(sampled), title('Sampled');
subplot(1,3,3), imshow(quantized), title('Quantized');
```

2. Brightness Transformation using Linear and Non-Linear Techniques

```
clc;

clear;

img = imread('image.jpg');
gray = rgb2gray(img);

linear = gray + 50;          % Linear
gamma = imadjust(gray,[],[],0.5); % Non-linear

subplot(1,3,1), imshow(gray), title('Original');
subplot(1,3,2), imshow(linear), title('Linear');
subplot(1,3,3), imshow(gamma), title('Non-Linear');
```

3. Local Pre-processing Techniques for Noise Reduction

```
clc; clear;

img = imread('C:\Users\LENOVO\Downloads\vijay8d.jpg');
gray = rgb2gray(img);

noisy = imnoise(gray,'salt & pepper',0.02);
filtered = medfilt2(noisy,[3 3]);
```

ALL EXECUTED PROGRAMS

```
subplot(1,3,1), imshow(gray), title('Original');  
subplot(1,3,2), imshow(noisy), title('Noisy');  
subplot(1,3,3), imshow(filtered), title('Filtered');
```

4. Thresholding-Based Image Segmentation

```
clc;  
clear;
```

```
img = rgb2gray(imread('C:\Users\LENOVO\Downloads\vijay8d.jpg'));
```

```
bw=img>100; % For older MATLAB
```

```
subplot(1,2,1), imshow(img), title('Grayscale');  
subplot(1,2,2), imshow(bw), title('Threshold Image');
```

5. Edge Detection Techniques for Image Segmentation

```
clc; clear;
```

```
img = imread('image.jpg');
```

```
gray = rgb2gray(img);
```

```
sobel = edge(gray,'sobel');
```

```
canny = edge(gray,'canny');
```

```
subplot(1,3,1), imshow(gray), title('Original');  
subplot(1,3,2), imshow(sobel), title('Sobel Edge');  
subplot(1,3,3), imshow(canny), title('Canny Edge');
```

6. Region-Based Segmentation using Connected Components

```
clc;  
clear;
```

```
img =rgb2gray(imread('C:\Users\LENOVO\Downloads\vijay8d.jpg'));
```

```
bw = img>100; % Threshold
```

```
label= bwlabel(bw); % Color labeling
```

```
subplot(1,3,1), imshow(img), title('Grayscale Image');
```

```
subplot(1,3,2), imshow(bw), title('Binary Image');  
subplot(1,3,3), imshow(label2rgb(label)), title('Connected Components');
```

7. Basic Gray Level Transformations

```
clc;  
clear;  
img = imread('image.jpg');  
gray = rgb2gray(img);  
  
neg = 255 - gray;  
log_img = uint8(255*mat2gray(log(1+double(gray))));  
power = imadjust(gray,[],[],2);
```

```
subplot(2,2,1), imshow(gray), title('Original');  
subplot(2,2,2), imshow(neg), title('Negative');  
subplot(2,2,3), imshow(log_img), title('Log');  
subplot(2,2,4), imshow(power), title('Power');
```

8. Histogram Equalization and Contrast Enhancement

```
clc;  
clear;  
  
img = imread('C:\Users\LENOVO\Downloads\vijay8d.jpg');  
gray = rgb2gray(img);  
  
eq = histeq(gray);  
  
subplot(2,2,1), imshow(gray), title('Original');  
subplot(2,2,2), imhist(gray), title('Histogram');  
subplot(2,2,3), imshow(eq), title('Equalized');  
subplot(2,2,4), imhist(eq), title('Eq Histogram');
```

9. Image Smoothing using Spatial Filters

```
clc;  
clear;  
img = imread('image.jpg');
```

```
gray = rgb2gray(img);

avg = imfilter(gray, fspecial('average',[3 3]));    % Average filter
gauss = imgaussfilt(gray,2);                      % Gaussian filter

subplot(1,3,1), imshow(gray), title('Original');
subplot(1,3,2), imshow(avg), title('Average');
subplot(1,3,3), imshow(gauss), title('Gaussian');
```

10. Image Sharpening using Spatial Filters

```
clc;
clear;

img = imread('image.jpg');
gray = rgb2gray(img);

lap = imfilter(gray, fspecial('laplacian',0.2)); % Laplacian
sharp = imsubtract(gray, lap);                 % Sharpening

subplot(1,3,1), imshow(gray), title('Original');
subplot(1,3,2), imshow(lap,[]), title('Laplacian');
subplot(1,3,3), imshow(sharp), title('Sharpened');
```

11. Fourier Transform for Image Processing and Frequency Domain Analysis

```
clc;
clear;

img = imread('image.jpg');
gray = rgb2gray(img);

f = fft2(double(gray));    % Fourier transform
fshift = fftshift(f);      % Shift center
mag = log(1+abs(fshift));   % Magnitude

subplot(1,2,1), imshow(gray), title('Original');
subplot(1,2,2), imshow(mag,[]), title('Fourier Spectrum');
```

12. Homomorphic Filtering for Illumination Correction

```
clc;

clear;

img = imread('image.jpg');
gray = im2double(rgb2gray(img));

log_img = log(1+gray);
F = fftshift(fft2(log_img));

[M,N] = size(gray);
[X,Y] = meshgrid(1:N,1:M);
D = (X-N/2).^2 + (Y-M/2).^2;

H = 1 - exp(-D/(2*30^2)); % High-pass filter
G = H .* F;

out = real(ifft2(ifftshift(G)));
result = mat2gray(exp(out)-1);

subplot(1,2,1), imshow(gray), title('Original');
subplot(1,2,2), imshow(result), title('Homomorphic');
```

13. Binary Morphological Operations: Dilation and Erosion

```
clc;

clear;

img = imread('image.jpg');
gray = rgb2gray(img);

bw = im2bw(gray, graythresh(gray)); % Binary (works in all versions)
se = strel('square',3);

dil = imdilate(bw,se); % Dilation
ero = imerode(bw,se); % Erosion
```

```
subplot(1,3,1), imshow(bw), title('Binary');  
subplot(1,3,2), imshow(dil), title('Dilation');  
subplot(1,3,3), imshow(ero), title('Erosion');
```

14. Gray-Scale Morphological Operations: Dilation and Erosion

```
clc;  
clear;  
img = imread('image.jpg');  
gray = rgb2gray(img);  
  
se = strel('square',3);  
  
dil = imdilate(gray,se); % Gray dilation  
ero = imerode(gray,se); % Gray erosion  
  
subplot(1,3,1), imshow(gray), title('Grayscale');  
subplot(1,3,2), imshow(dil), title('Gray Dilation');  
subplot(1,3,3), imshow(ero), title('Gray Erosion');
```

15. Morphological Segmentation using Watershed Algorithm

```
clc;  
clear;  
img = imread('image.jpg');  
gray = rgb2gray(img);  
  
gmag = imgradient(gray); % Gradient  
L = watershed(gmag); % Watershed  
  
seg = gray;  
seg(L==0) = 255; % Mark boundaries  
  
subplot(1,3,1), imshow(gray), title('Original');  
subplot(1,3,2), imshow(gmag,[]), title('Gradient');  
subplot(1,3,3), imshow(seg), title('Watershed');
```